Exercices de programmation

Julien Reichert

2026/2027

Ce document est une liste d'exercices de programmation faisant office de « minimum syndical » en termes de maîtrise de la syntaxe des langages de programmation au programme de classes préparatoires (Python pour le tronc commun, OCaml pour l'option informatique et l'informatique de MP2I, C pour l'informatique de MP2I et SQL pour le tronc commun et l'informatique de MP2I). Il est écrit spécialement pour la classe de MP2I, mais peut être utilisé par d'autres classes en adaptant la chronologie à la progression prévue par le programme officiel, ainsi que le langage.

Les exercices qui y figurent sont à faire chez soi, en faire la correction en classe n'est pas systématique, bien qu'on puisse imaginer des séances spécialement dédiées à cette liste.

Il est possible que des exercices aient des prérequis mathématiques. Normalement, ils sont placés dans une période où l'avancée du cours dans les deux matières est suffisante.

La liste des exercices a été intégralement rénovée en 2026/2027. Celle des quatre années précédentes est encore disponible et il est fortement suggéré de faire les anciens exercices en parallèle aux périodes indiquées.

Table des matières

1 Les exercices à maîtriser avant la rentrée [Python] 3

2 Les exercices de la première période [C] 4

3 Les exercices des vacances de la Toussaint [C] 5

4 Les exercices de la deuxième période [Ocaml] 6

5 Les exercices des vacances de Noël [Ocaml] 7

6 Les exercices de la troisième période [C et OCaml] 8

7 Les exercices des vacances d'hiver [C et OCaml] 9

1 Les exercices à maîtriser avant la rentrée [Python]

Toutes les fonctions de cette section sont à écrire en Python.

Exercice 0 : Écrire une fonction `pair` prenant en argument un entier et déterminant s'il est pair.

Exercice 1 : Écrire une fonction `avantdernier` prenant en argument une liste de taille au moins deux (pas à vérifier) et renvoyant son avant-dernier élément.

Exercice 2 : Écrire une fonction `premier` prenant en argument un entier naturel et déterminant s'il est premier.

Exercice 3 : Écrire une fonction `sip` prenant en argument une liste d'entiers et renvoyant la somme des entiers aux indices qui sont des nombres premiers.

Exercice 4 : Écrire une fonction `nbpf` prenant en argument une liste et renvoyant le nombre d'occurrences maximal d'un de ses éléments.

Exercice 5 : Écrire une fonction `nbnbpf` prenant en argument une liste et renvoyant le nombre d'éléments différents dont le nombre d'occurrences est le nombre d'occurrences maximal d'un élément de la liste.

Exercice 6 : Écrire une fonction `sommesansequilibre` prenant en argument deux listes de nombres et renvoyant la liste obtenue en additionnant les éléments des deux listes de mêmes indices. Si la taille d'une liste est strictement inférieure à la taille de l'autre, les éléments excédentaires sont reportés comme si la liste plus courte avait été complétée par des zéros (mais on ne doit pas la muter en la complétant).

Exercice 7 : Écrire une fonction `espaces` prenant en argument une chaîne de caractères et renvoyant le nombre d'espaces qui y figurent.

Exercice 8 : Écrire une fonction `chiffresdiff` prenant en argument une chaîne de caractères et renvoyant le nombre de chiffres différents qui y figurent.

Exercice 9 : Écrire une fonction `sousmot` prenant en argument deux chaînes de caractères et déterminant si la première est un sous-mot de l'autre, c'est-à-dire qu'on trouve les caractères de la première chaîne dans le même ordre mais pas nécessairement de manière consécutive dans la deuxième chaîne.

2 Les exercices de la première période [C]

Toutes les fonctions de cette section sont à écrire en C.

Exercice 10 : Écrire une fonction `multarr` prenant en argument deux flottants positifs et renvoyant le produit des deux arrondi à l'entier inférieur et exprimé en tant qu'entier. On ne vérifiera ni la positivité ni l'absence de dépassement arithmétique.

Exercice 11 : Écrire une fonction `moislong` prenant en argument un entier entre `1` et `12` correspondant à un numéro de mois du calendrier grégorien et déterminant si le mois a 31 jours.

Exercice 12 : Écrire une fonction `milieu` prenant en argument une chaîne de caractères supposée non vide (pas à vérifier) et renvoyant son caractère du milieu. Si la taille est paire, on renverra celui de gauche parmi les deux caractères du milieu.

Exercice 13 : Écrire une fonction `ecartabsmax` prenant en argument un tableau d'entiers de taille au moins deux (pas à vérifier) et sa taille et renvoyant l'écart entre la plus grande valeur absolue d'un élément du tableau et la plus petite valeur absolue d'un élément du tableau.

Exercice 14 : Écrire une fonction `extraire` prenant en argument une chaîne de caractères et renvoyant l'entier obtenu en collectant tous les chiffres de la chaîne à l'exception des zéros initiaux, en ignorant tous les autres caractères même les signes moins. Si le nombre de chiffres est de dix ou plus, on déclenchera une erreur, et s'il n'y en a pas on renverra `0`.

Exercice 15 : Écrire une fonction `nb1` prenant en argument un entier positif et renvoyant le nombre de `1` dans son écriture en binaire.

Exercice 16 : Écrire une fonction `prodmin` prenant en argument deux tableaux de flottants non vides et leur taille commune et renvoyant le plus petit produit que l'on peut obtenir en multipliant un élément du premier tableau avec l'élément au même indice du deuxième. On ne vérifiera pas que les tableaux sont non vides et de même taille.

Exercice 17 : Écrire une fonction `moyent` prenant en argument un tableau d'entiers positifs (pas à vérifier) et sa taille et déterminant si sa moyenne est un nombre entier. On prendra garde à limiter les risques de dépassement arithmétique.

Exercice 18 : Écrire une fonction `bezout` prenant en argument deux entiers naturels non nuls et renvoyant un couple de Bézout pour ces deux entiers.

Exercice 19 : Écrire une fonction `base` prenant en argument deux entiers positifs et renvoyant la conversion du premier entier dans la base du deuxième, qui est forcément entre `2` et `10` inclus. La réponse sera stockée dans un tableau en fournissant également la taille d'une manière au choix. L'ordre des éléments dans le tableau sera au choix mais la fonction de test devra être cohérente avec cet ordre.

3 Les exercices des vacances de la Toussaint [C]

Toutes les fonctions de cette section sont à écrire en C.

Exercice 20 : Écrire une fonction `nbindnb` prenant en argument un tableau d'entiers et sa taille et renvoyant le nombre d'indices où se situent dans le tableau un nombre dont le nombre d'occurrences est égal à l'indice.

Exercice 21 : Écrire une fonction `nbnbnb` prenant en argument un tableau d'entiers et sa taille et renvoyant le nombre d'entiers strictement positifs égaux à leur nombre d'occurrences dans le tableau.

Exercice 22 : Écrire une fonction `nbsomindnb` prenant en argument un tableau d'entiers et sa taille et renvoyant le nombre d'entiers naturels égaux à la somme des indices de leurs occurrences dans le tableau.

Exercice 23 : Écrire une fonction `prodind` prenant en argument un tableau d'entiers et sa taille ainsi qu'un entier et renvoyant le produit des indices où cet entier figure (inutile de gérer les dépassements).

Exercice 24 : Écrire une fonction `cycle` prenant en argument une chaîne de caractères et un entier et renvoyant la chaîne obtenue en décalant la chaîne d'un nombre d'indices égal à l'entier en deuxième argument. Autrement dit, si l'entier vaut `3`, les trois premiers caractères sont envoyés à la fin. On supposera (inutile de le vérifier) que le décalage est un nombre entre zéro inclus et la taille de la chaîne exclue.

Exercice 25 : Écrire une fonction `cycles` prenant en argument une chaîne de caractères non vide (inutile de le vérifier) et renvoyant le tableau des chaînes obtenues par tous les décalages possibles (leur nombre est la taille de la chaîne).

Exercice 26 : Écrire une fonction `cyclemin` prenant en argument une chaîne de caractères et renvoyant la plus petite (dans l'ordre lexicographique) des chaînes obtenues par décalage de la chaîne en argument.

Exercice 27 : Écrire une fonction `trichaine` prenant en argument une chaîne de caractères et la mutant en permutant ses caractères afin qu'ils soient dans l'ordre croissant selon leur code ASCII (tous les caractères sont supposés être dans la table ASCII-8 bits).

Exercice 28 : Écrire une fonction `tibere` prenant en argument une chaîne de caractères en chiffres romains et renvoyant l'entier correspondant. On ne vérifiera pas que la chaîne correspond bien à un entier en chiffres romains.

Exercice 29 : Écrire une fonction `compresseromain` prenant en argument une chaîne de caractères et renvoyant la chaîne obtenue en convertissant toutes les suites de caractères correspondant à des chiffres romains dans la chaîne en le chiffre correspondant entouré de crochets. Quand un chiffre romain est rencontré, on capture le plus grand nombre de caractères consécutifs possible à partir de ce chiffre. Les autres caractères sont reportés tels quels. Pour simplifier, un caractère n'est interprété comme un chiffre romain que s'il est en capitales. Attention, la chaîne `"IVI"` pourra être rencontrée et il faudra renvoyer `"[4][1]"`.

4 Les exercices de la deuxième période [Ocaml]

Toutes les fonctions de cette section sont à écrire en OCaml.

Exercice 30 : Écrire une fonction `bord` prenant en argument un entier strictement positif et renvoyant le plus grand entier positif obtenu en multipliant par des puissances de `2` avant un dépassement arithmétique.

Exercice 31 : Écrire une fonction `dernier` prenant en argument une liste et renvoyant son dernier élément. Le comportement est au choix si la liste est vide.

Exercice 32 : Écrire une fonction `demipairs` prenant en argument une liste d'entiers et renvoyant une liste obtenue en extrayant les éléments pairs, puis en ne prenant que la moitié de ceux-ci selon l'alternance suivante : on garde, on ne garde pas, on garde, etc. Le travail peut se faire en une fois (c'est plus classe).

Exercice 33 : Écrire une fonction `changements` prenant en argument une liste d'entiers et renvoyant le nombre de fois où le maximum provisoire est battu. Pour une liste de taille inférieure ou égale à un, la réponse sera en particulier `0`.

Exercice 34 : Écrire une fonction `bp` prenant en argument une liste et renvoyant le nombre d'éléments qui seraient bien placés dans la version triée de la liste.

Exercice 35 : Écrire une fonction `duel` prenant en argument deux listes de même taille (inutile de le vérifier) et déterminant si, en comparant les éléments aux mêmes positions dans les deux listes, il y a strictement plus de fois où l'élément de la première est strictement supérieur à l'élément de la deuxième que de fois où l'élément de la première est strictement inférieur à l'élément de la deuxième.

Exercice 36 : Écrire une fonction `nbchf` prenant en argument un entier strictement positif (inutile de le vérifier) et renvoyant son nombre de chiffres différents.

Exercice 37 : Écrire une fonction `entrezeros` prenant en argument un entier positif (inutile de le vérifier) et renvoyant l'entier que l'on peut former entre les deux premiers zéros de l'entier (en base dix). S'il n'y en a pas deux, on renverra `0`.

Exercice 38 : Écrire une fonction `maxentrezeros` prenant en argument un entier positif (inutile de le vérifier) et renvoyant le plus grand entier que l'on peut former entre des zéros de l'entier (en base dix). On tiendra compte des chiffres avant le premier zéro ou après le dernier zéro et on renverra `0` si le nombre est nul.

Exercice 39 : Écrire une fonction `maxvraimententrezeros` prenant en argument un entier positif (inutile de le vérifier) et renvoyant le plus grand entier que l'on peut former entre des zéros de l'entier (en base dix). On ne tiendra pas compte des chiffres avant le premier zéro ou après le dernier zéro, donc s'il n'y a pas deux zéros, on renverra `0`.

5 Les exercices des vacances de Noël [Ocaml]

Toutes les fonctions de cette section sont à écrire en OCaml.

Exercice 40 : Écrire une fonction `julien` prenant en argument une chaîne de caractères et déterminant si tous ses caractères sont des lettres (capitales ou non) constituant le prénom du professeur d'informatique.

Exercice 41 : Écrire une fonction `kikoo` prenant en argument une chaîne de caractères et déterminant si les caractères qui sont des lettres alternent entre capitale et bas de casse (la première peut être capitale ou non).

Exercice 42 : Écrire une fonction `quarante_deux` prenant en argument une chaîne de caractères et déterminant si le nombre d'occurrences du facteur `"42"` et du facteur `"quarante-deux"` sont identiques.

Exercice 43 : Écrire une fonction `monotone` prenant en argument un tableau et déterminant s'il est monotone.

Exercice 44 : Écrire une fonction `repetepas` prenant en argument une chaîne de caractères et renvoyant la chaîne obtenue si, lorsque plusieurs caractères consécutifs sont identiques, un seul est conservé.

Exercice 45 : Écrire une fonction `tousminis` prenant en argument un tableau de tableaux tous de même taille (inutile de le vérifier) et renvoyant le tableau où à chaque indice figure le minimum des éléments à l'indice correspondant parmi tous les tableaux.

Exercice 46 : Écrire une fonction `palindr` prenant en argument une chaîne de caractères et renvoyant la taille maximale, limitée à la moitié de la taille de la chaîne arrondie par excès, d'un préfixe de la chaîne dont le miroir est un suffixe de la chaîne.

Exercice 47 : Écrire une fonction `intrus` prenant en argument une liste de tableaux d'entiers dont la somme est toujours la même sauf pour l'un d'entre eux (inutile de le vérifier), et renvoyant le tableau ayant une somme différente. S'il y a strictement moins de trois tableaux, on déclenchera une erreur.

Exercice 48 : Écrire une fonction `perimetre` prenant en argument un tableau de couples de flottants représentant les sommets d'un polygone convexe (pas à vérifier) et renvoyant le périmètre du polygone.

Exercice 49 : Écrire une fonction `echo` prenant en argument une chaîne de caractères et renvoyant la chaîne obtenue en extrayant ce qui figure après le premier facteur `"di"` (en minuscules) ou après le premier `"cri"` (en minuscules). On cherchera le premier de ces deux facteurs si les deux apparaissent. Si le facteur est `"cri"`, les lettres extraites doivent être mises en capitales.

6 Les exercices de la troisième période [C et OCaml]

Toutes les fonctions de cette section sont à écrire en C et en OCaml. Si un tableau doit être un argument, sa taille est à mettre en argument aussi en C uniquement. Pour cette section et les suivantes, il est possible que le type des arguments (ou des valeurs de retour) dépende du langage. Attention à bien lire la consigne, car il y aura souvent du polymorphisme en OCaml.

Exercice 50 : Écrire une fonction `ensemble_egal` prenant en argument deux tableaux d'entiers (en C) ou deux listes (en OCaml) et déterminant si leurs éléments sont les mêmes sans tenir compte du nombre d'occurrences.

Exercice 51 : Écrire une fonction `sequence_egale` prenant en argument deux tableaux d'entiers (en C) ou deux listes (en OCaml) et déterminant si leurs éléments sont les mêmes en tenant compte du nombre d'occurrences.

Exercice 52 : Écrire une fonction `plus_proche` prenant en argument `tabtab`, un tableau de tableau de flottants tous de même taille et `tabref`, un autre tableau de flottants, également de la taille commune, et renvoyant le tableau dans `tabtab` qui minimise la distance à `tabref` définie comme la somme des valeurs absolues des écarts entre deux éléments de même indice.

Exercice 53 : Écrire une fonction `tri_straline` prenant en argument un tableau d'entiers (en C) ou une liste (en OCaml) et renvoyant sa version triée selon l'algorithme de tri qui incorpore tous les éléments qui laissent la séquence croissante et qui ignore les autres. Autrement dit, il s'agit d'extraire la sous-séquence croissante collectant le plus d'indices possibles et le plus tôt possible.

Exercice 54 : Écrire une fonction `maxocc1` prenant en argument un tableau d'entiers et renvoyant le maximum pour tous les entiers `n` du nombre d'occurrences cumulées dans le tableau de `n-1`, `n` et `n+1`.

Exercice 55 : Écrire une fonction `maxocc1i` prenant en argument un tableau d'entiers (en C) ou une liste (en OCaml) et renvoyant le maximum pour tous les indices `i` du nombre d'occurrences cumulées dans le tableau des entiers aux indices `i-1`, `i` et `i+1`, sachant qu'en cas d'égalité d'au moins deux des trois éléments de la séquence le nombre d'occurrences n'est compté qu'une fois.

Exercice 56 : Écrire une fonction `sommeuniques` prenant en argument un tableau d'entiers et sa taille et renvoyant la somme des entiers n'apparaissant qu'une fois dans le tableau.

Exercice 57 : Écrire une fonction `rpz` prenant en argument une chaîne de caractères et déterminant si le nombre d'occurrences dans cette chaîne de chaque lettre, sans tenir compte de la casse, du nom du plus beau département de France (son numéro est le numéro de la question) est identique.

Exercice 58 : Écrire une fonction `purge` prenant en argument un tableau d'entiers positifs et sa taille et renvoyant le tableau privé de ses éléments dont le nombre d'occurrences est minimal ou maximal parmi les nombres d'occurrences des éléments. On fournira également la taille d'une manière au choix en C.

Exercice 59 : Écrire une fonction `score_bridge` prenant en argument une chaîne de caractères décrivant un contrat de bridge (par exemple `"3SA"` ou `"5PXX"`), un entier donnant le nombre de levées du déclarant et un booléen précisant la vulnérabilité du déclarant et renvoyant le score du preneur associé à ces informations.

7 Les exercices des vacances d'hiver [C et OCaml]

Toutes les fonctions de cette section sont à écrire en C et en OCaml. Si un tableau doit être un argument, sa taille est à mettre en argument aussi en C uniquement.

Exercice 60 : Écrire une fonction `pas_choisi` prenant en argument deux tableaux d'entiers de taille dix (pas à vérifier) `t1` et `t2` et renvoyant la plus petite somme que l'on peut obtenir en choisissant quatre à six indices possibles et en additionnant les éléments de `t1` aux indices choisis et les éléments de `t2` aux autres.

Exercice 61 : Écrire une fonction `desordre` prenant en argument un tableau de couples de chaînes de caractères et déterminant s'il existe deux indices `i` et `j` pour lesquels le couple à l'indice `i` du tableau est strictement inférieur au couple à l'indice `j` selon l'ordre lexicographique des couples mais lui est strictement supérieur si on considère la concaténation des deux chaînes du couple.

Exercice 62 : Écrire une fonction `incomparables` prenant en argument un tableau de couples de chaînes de caractères (en C) ou une liste de couples (en OCaml) et renvoyant le nombre de fois où deux éléments sont incomparables selon l'ordre produit.

Exercice 63 : Écrire une fonction `maxtotal` prenant en argument un tableau de tableaux de couples d'entiers et renvoyant la plus grande somme sur un tableau des différences entre les deux éléments des couples.

Exercice 64 : Écrire une fonction `interpol4` prenant en argument deux tableaux de flottants de taille quatre (pas à vérifier), le premier correspondant à des réels distincts deux à deux (pas à vérifier) et le deuxième correspondant à l'image des réels au même indice par une unique fonction polynomiale de degré au plus trois (interpolation de Lagrange), et renvoyant les coefficients de cette fonction polynomiale sous la forme d'un tableau dont le coefficient constant est en premier.

Exercice 65 : Écrire une fonction `rotuniques` prenant en argument un tableau de tableaux d'entiers et renvoyant le nombre de tableaux différents à rotation des indices près

Exercice 66 : Écrire une fonction `permuniques` prenant en argument un tableau de tableaux d'entiers et renvoyant le nombre de tableaux différents à permutation des indices près.

Exercice 67 : Écrire une fonction `hopla` prenant en argument une chaîne de caractères contenant uniquement des lettres et des espaces (pas à vérifier), et déterminant si la combinaison de trois lettres consécutives dans un mot (suite de lettres sans espaces) la plus fréquente (éventuellement à égalité) est `"sch"`.

Exercice 68 : Écrire une fonction `suffmajo` prenant en argument une chaîne de caractères contenant uniquement des lettres et des espaces (pas à vérifier) et un entier naturel non nul (pas à vérifier), et renvoyant le suffixe, du nombre de lettres égal à l'entier en argument, le plus fréquent d'un mot (n'importe lequel en cas d'égalité). On ignorera les mots trop courts.

Exercice 69 : Écrire une fonction `mouvements_backgammon` prenant en argument l'état d'un plateau de backgammon en tant que tableau de vingt-quatre entiers selon l'encodage zéro pour une case vide et un autre entier pour un nombre de jetons égal à la valeur absolue de l'entier et appartenant au joueur J si l'entier est positif et à son adversaire sinon, et deux entiers correspondant chacun à un lancer de dés et renvoyant le tableau des cases accessibles par l'utilisation d'un des dés fournis (donc un mouvement partiel, pas le tour entier) par le joueur J. On considère pour simplifier la situation où les jetons ne peuvent pas encore être retirés du jeu et aucun jeton du joueur J n'a été capturé.